UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO FACULTAD DE INGENIERÍA DE CIENCIAS COMPUTACIONALES Y TELECOMUNICACIONES



Proyecto 2

Cifrador de Hill y Criptoanalisis Gauss-Jordan (GRUPO D)

INTEGRANTES:

Alurralde Nelson	221043551
Guzman Montalvan Marcos Lehonti	220030081
Ponce Yepez Andres Reyes	220002223
Villarroel Siancas Marcos Alexander	216058368
Zarate Mollo Josué	220030561

MATERIA: Criptografía y Seguridad - SA

DOCENTE: Ing. Pérez Delgadillo Shirley Eulalia

SEMESTRE: 2/2026

Santa Cruz de la Sierra — 2026

1. Introducción teórica y análisis criptográfico....3
1.1. El cifrado de Hill....3
1.2. Aritmética modular e inversos....3
1.3. El método de Gauss-Jordan....4
1.4. Criptoanálisis de Hill con Gauss-Jordan....4
Si P no es invertible mod 26, el ataque puede intentarse con otros n bloques del texto conocido, hasta encontrar una P invertible....4
1.5. Análisis criptográfico....4
2. Explicación detallada del algoritmo implementado....6
2.1. Alfabeto de Hill....6
2.2. Cifrado....6
2.3. Descifrado....7
2.4. Inversión de la matriz clave (Gauss-Jordan)....7
2.5. Criptoanálisis por Gauss-Jordan....7
2.6. Validación de invertibilidad....7
2.7. Arquitectura del software....7
3. Casos de prueba y depuración paso a paso....9
3.1. Matriz clave y determinante....9
3.2. Cifrado bloque por bloque....9
3.3. Inversión de K con Gauss-Jordan (23 pasos)....9
3.4. Descifrado bloque por bloque....10
3.5. Criptoanálisis: recuperación de K a partir de P y C....11
3.6. Caso 3×3....11
3.7. Casos de error (mensajes reales del programa)....11
3.8. Pruebas automáticas (opción 6)....12
4. Evidencia de ejecución....14
4.1 Versión de consola (aplicación principal)....14
4.2 Versión con interfaz gráfica (C++Builder, VCL)....15
5. Reflexión sobre limitaciones y posibles mejoras....17
6. Conclusiones....19
7. Referencias....20
Anexos: trazas completas de Gauss-Jordan....21
Anexo A. Inversa de la K oficial 4×4 (23 pasos)....22
Anexo B. Inversión de P en el criptoanálisis del caso oficial (23 pasos)....24
Anexo C. Inversa del caso 3×3 (13 pasos)....26
Anexo D. Matriz singular: K = [[1,2,3],[2,4,6],[5,7,9]] (opción 3)....27

1. Introducción teórica y análisis criptográfico

El cifrado de Hill (Lester S. Hill, 1929) es lineal, por lo que se rompe con un ataque de texto plano conocido: este proyecto lo cifra, lo descifra y recupera la clave con Gauss-Jordan módulo 26.



1.1. El cifrado de Hill

Hill fue el primer cifrador poligráfico clásico basado en álgebra lineal. A diferencia de los cifrados monoalfabéticos (César, sustitución simple), cifra bloques de n letras a la vez: cada bloque es un vector columna P que se multiplica por una matriz clave K de tamaño n×n, con aritmética módulo 26.

$$C = K \cdot P \pmod{26}$$

Para descifrar se necesita la inversa de K en Z_{26}:

$$P = K^{-1} \cdot C \pmod{26}$$

K^{-1} existe solo si el determinante de K es coprimo con 26, es decir, mcd(det K, 26) = 1. Como 26 = 2 × 13, el determinante no puede ser par ni múltiplo de 13.

1.2. Aritmética modular e inversos

En Z_{26} la suma, la resta y la multiplicación funcionan de forma habitual, pero la división no siempre es posible. Un número a tiene inverso multiplicativo módulo 26 solo si mcd(a, 26) = 1. Ese inverso puede calcularse con el algoritmo extendido de Euclides o, al ser el módulo tan pequeño, probando los valores del 1 al 25; la implementación usa esta búsqueda exhaustiva (`MatrizMod26::inversoMod`). Como 26 no es primo, Z_{26} no es un cuerpo, y esto condiciona todo el álgebra de matrices que sigue.

1.3. El método de Gauss-Jordan

Gauss-Jordan lleva una matriz a su forma escalonada reducida mediante tres operaciones elementales entre filas: intercambiar dos filas, multiplicar una fila por un escalar con inverso, y sumar a una fila un múltiplo de otra. Para invertir una matriz A se parte de la matriz aumentada [A | I] y se opera hasta obtener [I | A^{-1}].

En Z_{26}, cada pivote debe tener inverso módulo 26. Si en una columna ningún candidato lo tiene, la matriz no es invertible.

1.4. Criptoanálisis de Hill con Gauss-Jordan

El cifrado de Hill es vulnerable a un ataque de texto plano conocido (Known-Plaintext Attack, KPA) debido a la linealidad del sistema. Si un atacante conoce *n* bloques de texto plano y sus correspondientes bloques de texto cifrado, es posible recuperar la matriz clave *K* resolviendo un sistema de ecuaciones lineales en $\mathbb{Z}_{26}$.

Dado un conjunto de vectores columna de texto plano $P_1, P_2, ..., P_n$ y sus correspondientes vectores cifrados $C_1, C_2, ..., C_n$, se pueden formar dos matrices de tamaño $n \times n$ colocando los vectores como columnas:

$$P = [\ P_1 \mid P_2 \mid \ldots \mid P_n\ ]$$

$$C = [\ C_1 \mid C_2 \mid \ldots \mid C_n\ ]$$

La relación entre las matrices está dada por la ecuación matricial:

$$C = K \cdot P \pmod{26}$$

Si la matriz de texto plano *P* es invertible en $\mathbb{Z}_{26}$ (es decir, $\gcd(\det P, 26) = 1$), se puede calcular su inversa P^{-1} aplicando el método de eliminación de Gauss-Jordan sobre la matriz aumentada $[P \mid I]$ módulo 26.

Una vez obtenida P^{-1}, la matriz clave *K* se recupera directamente multiplicando la matriz cifrada por la inversa del texto plano:

$$K = C \cdot P^{-1} \pmod{26}$$

Si P no es invertible mod 26, el ataque puede intentarse con otros n bloques del texto conocido, hasta encontrar una P invertible.

1.5. Análisis criptográfico

- **Fortalezas.** Hill difunde la información dentro del bloque (en general, cada letra cifrada depende de todas las letras del bloque, si la fila de K no tiene ceros), lo que lo hace resistente al análisis de frecuencias de monogramas.

- **Espacio de claves.** Hay $26^{16} \approx 4{,}4 \times 10^{22}$ matrices 4×4; las invertibles mod 26 son 12 303 585 972 327 392 870 400 ($\approx 1{,}2 \times 10^{22}$, un 28 %), calculadas como (invertibles mod 2 = 20 160) × (invertibles mod 13 $\approx 6{,}1 \times 10^{17}$). Por tanto, la fuerza bruta no es práctica.
- **Debilidades.** Presenta diversas vulnerabilidades estructurales:

  (a) **linealidad**: un ataque de texto plano conocido mediante $K = C \cdot P^{-1} \bmod 26$ permite recuperar la clave necesitando solo n bloques (n·n letras, 16 para 4×4) y que P sea invertible mod 26;
  (b) bloques de texto plano iguales dan bloques cifrados iguales (análogo al modo ECB);
  (c) la difusión ocurre solo dentro del bloque, no entre bloques;
  (d) **ausencia de confusión**: carece de componentes no lineales.
- **Comparativa de cifrados clásicos.**

Cifrador	**Unidad de procesamiento**	**Vulnerabilidad principal**
César / Afín	Letra a letra	Análisis de frecuencias de monogramas
Vigenère	Letra a letra con clave repetida	Examen Kasiski e índice de coincidencia
Hill	Bloques de n letras	Ataque de texto plano conocido (linealidad)

- **Impacto.** Hill introdujo el álgebra lineal en la criptografía; su ruptura muestra por qué los cifrados modernos incorporan componentes no lineales (como las cajas S de AES).

2. Explicación detallada del algoritmo implementado

La aplicación es un programa de consola en C++ estándar (g++, `-std=c++11`, solo librería estándar) que separa el núcleo matemático (`core/`) del menú (`main.cpp`).

2.1. Alfabeto de Hill

Se usa la tabla fija de la Tabla 1, implementada en `AlfabetoHill::letraANumero` y `AlfabetoHill::numeroALetra`.

Letra	A	B	C	D	E	F	G	H	I	J	K	L	M
Valor	5	23	2	20	10	15	8	4	18	25	0	16	13

Letra	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Valor	7	3	1	19	6	12	24	21	17	14	22	11	9

Tabla 1. Alfabeto de Hill utilizado (correspondencia letra-valor).

El enunciado no entrega la matriz clave ni el alfabeto. Se adoptó este alfabeto porque con él el par DELAYOPERATIONSU → JCOWZLVBDVLEQMXC determina una única K (det P = 15, invertible mod 26). Con el alfabeto estándar A=0…Z=25 el mismo par no permite recuperar K (det P = 24, mcd(24, 26) = 2).

Un alfabeto distinto del estándar no altera la seguridad matemática del esquema: es una permutación fija de las 26 clases residuales. Sí hace que cualquier tabla de referencia que asuma A=0…Z=25 dé resultados distintos, por lo que alfabeto, matrices, cifrado y Gauss-Jordan deben ser consistentes con la Tabla 1.

Antes de cifrar, `AlfabetoHill::limpiar` pasa el texto a mayúsculas y descarta todo lo que no sea A–Z (incluidas Ñ y acentos). Si la longitud no es múltiplo de n, `AlfabetoHill::rellenar` agrega la letra X; la consola pregunta antes de rellenar.

2.2. Cifrado

`HillCipher::cifrar` limpia y rellena el texto, lo divide en bloques de n letras (`dividirBloques`), convierte cada bloque en vector (`textoAVector`), calcula C = K·P mod 26 con `MatrizMod26::multiplicar` y reconstruye las letras con `vectorATexto`. La consola se niega a cifrar si K no es invertible, porque el mensaje no podría descifrarse.

2.3. Descifrado

`HillCipher::descifrar` obtiene K^{-1} con `GaussJordan::invertir` y aplica $P = K^{-1} \cdot C \bmod 26$ a cada bloque. Devuelve cadena vacía si K no es invertible.

2.4. Inversión de la matriz clave (Gauss-Jordan)

`GaussJordan::invertir` construye [K | I] y, para cada columna j = 1…n:

1. Busca una fila i ≥ j cuyo elemento en la columna j sea coprimo con 26 (si no hay ninguna, K no es invertible y devuelve matriz vacía).
2. Intercambia esa fila con la fila j.
3. Multiplica la fila j por el inverso modular del pivote (`inversoMod`), dejándolo en 1.
4. Para cada otra fila, resta (elemento de la columna j) × (fila j), todo mod 26.

Al terminar, la mitad izquierda es I y la derecha es K^{-1}. La sobrecarga con `std::vector<Paso>` registra cada operación (descripción y matriz aumentada resultante), que es lo que muestra la opción 3 del menú.

2.5. Criptoanálisis por Gauss-Jordan

`GaussJordan::recuperarClave(P, C)` calcula P^{-1} por el mismo mecanismo y devuelve $K = C \cdot P^{-1} \bmod 26$ (vacía si P no es invertible). La opción 4 del menú acepta P y C como matrices o como texto, y admite textos de más de n·n letras: prueba combinaciones de n bloques en orden hasta hallar una P invertible, informa qué bloques usó y verifica la K recuperada cifrando todo el texto plano y comparándolo con el criptograma. Corta a las 100000 combinaciones.

2.6. Validación de invertibilidad

`MatrizMod26::determinante` calcula det(K) mod 26 por expansión de cofactores (recursiva) y `MatrizMod26::esInvertible` verifica que mcd(det K, 26) = 1 usando `inversoMod`. La consola lo comprueba antes de cifrar, descifrar o invertir.

2.7. Arquitectura del software

Archivo	Responsabilidad
`main.cpp`	Menú de consola (1 Cifrar, 2 Descifrar, 3 Inversa con pasos, 4 Criptoanálisis, 5 Caso oficial, 6 Pruebas automáticas, 0 Salir). Pide datos, valida entradas e imprime; no reimplementa lógica.
`core/Tipos.h`	Constante `MOD = 26` y alias `Vec`, `Mat`.

Archivo	Responsabilidad
`core/AlfabetoHill`	Conversión letra↔número, limpieza y relleno.
`core/MatrizMod26`	Aritmética modular de matrices: `mod`, `identidad`, `multiplicar`, `determinante`, `esInvertible`, `inversoMod`. Incluye además traspuesta y adjunta como utilidades auxiliares (la adjunta permite calcular $K^{-1} = \det(K)^{-1} \cdot \text{adj}(K)$); el programa no las usa, porque la inversión se hace con Gauss-Jordan.
`core/GaussJordan`	`invertir` y `recuperarClave`, con y sin registro de pasos (`struct Paso`).
`core/HillCipher`	`cifrar`, `descifrar`, `textoAVector`, `vectorATexto`, `dividirBloques`.
`build.bat`	Compila con `g++ -std=c++11` listando los `.cpp` de `core/`.

3. Casos de prueba y depuración paso a paso

El caso oficial 4×4 reproduce exactamente el criptograma esperado, y las 10 pruebas automáticas del programa terminan con 0 fallos. Todos los datos de esta sección salen de ejecutar el programa.

3.1. Matriz clave y determinante

La matriz K del caso oficial está fija en `main.cpp` (función `claveOficial4`):

```
[  8   6   9   5 ]
[  6   9   5  10 ]
[  5   8   4   9 ]
[ 10   6  11   4 ]
```

det K mod 26 = 25 y mcd(25, 26) = 1, por lo que K es invertible; el inverso del determinante es 25 (25 · 25 = 625 = 24 · 26 + 1).

Nota de depuración. El enunciado menciona un determinante «coprimo (21)». Con la K y el alfabeto que reproducen el criptograma esperado el determinante es 25, también coprimo con 26, por lo que el caso de prueba es igualmente válido. El 21 no apareció en ninguna de las convenciones probadas: con A=0…Z=25, P no es invertible (det P = 24), así que ese par no permite calcular K.

3.2. Cifrado bloque por bloque

Texto DELAYOPERATIONSU (16 letras = 4 bloques de 4, sin relleno).

Bloque	Texto	P (vector)	C = K·P mod 26	Cifrado
1	DELA	[20 10 16 5]	[25 2 3 14]	JCOW
2	YOPE	[11 3 1 10]	[9 16 17 23]	ZLVB
3	RATI	[6 5 24 18]	[20 17 16 10]	DVLE
4	ONSU	[3 7 12 21]	[19 13 22 2]	QMXC

Tabla 2. Cifrado bloque por bloque.

Criptograma obtenido: JCOW ZLVB DVLE QMXC, idéntico al del enunciado. `HillCipher::cifrar` produce la misma cadena.

3.3. Inversión de K con Gauss-Jordan (23 pasos)

La opción 3 registra 23 pasos (la traza completa está en el Anexo A). El resumen por columna es:

Columna	Pivote	Inverso	Operaciones (pasos)
1	5 (fila 3)	21	F1↔F3; F1=21·F1; F2=F2−6·F1; F3=F3−8·F1; F4=F4−10·F1 (2–7)
2	15 (fila 2)	7	F2=7·F2; F1=F1−12·F2; F3=F3−14·F2; F4=F4−16·F2 (8–12)
3	9 (fila 3)	3	F3=3·F3; F1=F1−10·F3; F2=F2−17·F3; F4=F4−17·F3 (13–17)
4	25 (fila 4)	25	F4=25·F4; F1=F1−F4; F2=F2−F4; F3=F3−25·F4 (18–22)

En la columna 1 los elementos 8, 6 y 10 son pares y no tienen inverso mod 26; el primer candidato coprimo es el 5 (fila 3), por eso se intercambian F1 y F3. Cálculo a mano del paso 4: F1 = 21·[5 8 4 9] = [105 168 84 189] mod 26 = [1 12 6 7], igual que la salida del programa.

Resultado [I | K^{-1}]:

```
              [ 23  20   5   1 ]
K^-1 =        [  2  11  18   1 ]
              [  2  20   6  25 ]
              [ 25   2  22  25 ]
```

Verificación: K · K^{-1} mod 26 = I (4×4); el programa responde CORRECTO.

3.4. Descifrado bloque por bloque

Bloque	C (vector)	P = K^{-1}·C mod 26	Texto recuperado
1	[25 2 3 14]	[20 10 16 5]	DELA
2	[9 16 17 23]	[11 3 1 10]	YOPE
3	[20 17 16 10]	[6 5 24 18]	RATI
4	[19 13 22 2]	[3 7 12 21]	ONSU

Tabla 3. Descifrado bloque por bloque.

Texto recuperado: DELAYOPERATIONSU, idéntico al original.

3.5. Criptoanálisis: recuperación de K a partir de P y C

Se toman los cuatro bloques de texto plano y de criptograma como columnas:

```
P = [ 20  11   6   3 ]      C = [ 25   9  20  19 ]
    [ 10   3   5   7 ]          [  2  16  17  13 ]
    [ 16   1  24  12 ]          [  3  17  16  22 ]
    [  5  10  18  21 ]          [ 14  23  10   2 ]
```

det P mod 26 = 15, coprimo con 26, así que P es invertible y el ataque es aplicable. La inversión de P también toma 23 pasos (Anexo B) y da:

```
P^-1 = [ 13  12   9  15 ]
       [ 14  16  15  20 ]
       [ 17   7  18   6 ]
       [ 19  12   5  14 ]
```

$K = C \cdot P^{-1} \bmod 26$ reproduce exactamente la K de 3.1 (el programa responde `K recuperada == K original? SI`). Como P es invertible, ese par texto plano/criptograma determina K de forma única bajo el alfabeto de la Tabla 1.

Texto conocido más largo. Con el texto HOLHOLHOLAMUNDOXYZ (3×3) los primeros bloques son idénticos y P sale singular; el programa descarta combinaciones, usa los bloques 1, 4 y 6, recupera la K original y la verificación sale CORRECTO. Si el criptograma se altera, la K recuperada no reproduce el criptograma y el programa indica INCORRECTO. Si ninguna combinación da una P invertible (HOLHOLHOLHOL), avisa que no puede recuperar K.

3.6. Caso 3×3

K = [[3, 10, 20], [20, 9, 17], [9, 4, 15]], det K mod 26 = 11, inverso del determinante = 19.

Bloque	Texto	P (vector)	K·P mod 26	Cifrado
1	HOL	[4 3 16]	[24 15 2]	TFC
2	AMU	[5 13 21]	[19 2 22]	QCX
3	NDO	[7 20 3]	[21 7 6]	UNR

Criptograma: TFCQCXUNR; el descifrado devuelve HOLAMUNDO. La inversa (13 pasos, Anexo C) es:

```
K^-1 = [ 25  22  18 ]
       [ 15   9   1 ]
       [  7   0  15 ]
```

$K \cdot K^{-1} = I$, verificado por el programa. Este mismo texto HOLAMUNDO tiene P singular, por lo que con solo 9 letras el criptoanálisis no puede recuperar su K (ver sección 5).

3.7. Casos de error (mensajes reales del programa)

Situación	Mensaje del programa
Cifrar con K no invertible (opción 1)	`ERROR: no se puede cifrar con una K no invertible (el mensaje no podria descifrarse).`
Descifrar con K no invertible (opción 2)	`ERROR: no se puede descifrar con una K no invertible.`
Inversa de K singular (opción 3)	`K NO es invertible mod 26 (mcd(det,26) != 1).` y, en el paso 9, `Columna 3: ningun elemento (desde la fila 3) es coprimo con 26; A no es invertible mod 26`
Descifrar con longitud no múltiplo de n	`ERROR: la longitud del criptograma (5) no es multiplo de n=4.`
Cifrar con longitud no múltiplo de n	`La longitud (10) no es multiplo de n=4; faltan 2 letra(s).` y pregunta `Rellenar con 'X'? (s/n):`; si se responde n: `Cifrado cancelado.`
Criptoanálisis con textos inválidos	`ERROR: ambos textos deben tener la misma longitud, multiplo de n=3 y de al menos 9 letras (n bloques de n letras).`
Criptoanálisis sin P invertible	`ERROR: ninguna combinacion de bloques da una P invertible mod 26; no se puede recuperar K. Use otro texto conocido.`
Texto vacío al cifrar (opción 1)	`ERROR: el texto esta vacio (solo se admiten letras A-Z).`

En la prueba de la matriz singular se usó K = [[1,2,3],[2,4,6],[5,7,9]] (det = 0); su traza está en el Anexo D.

3.8. Pruebas automáticas (opción 6)

N.º	Prueba	Resultado
1	Caso oficial 4×4: DELAYOPERATIONSU → JCOWZLVBDVLEQMXC	OK
2	Cifrar → descifrar 4×4 devuelve el original	OK
3	Caso 3×3: K invertible y cifrar → descifrar (HOLAMUNDO → TFCQCXUNR)	OK
4	Inversa 4×4: $K \cdot K^{-1} = I$	OK

N.º	Prueba	Resultado
5	Criptoanálisis 4×4: K recuperada = K original	OK
6	Matriz singular: se detecta y las operaciones avisan sin romper	OK
7	Validaciones: texto vacío y longitud no múltiplo de n	OK
8	Cifrar con K singular devuelve vacío	OK
9	Criptoanálisis con texto largo cuyos primeros bloques dan P singular	OK
10	Texto largo sin ninguna P invertible: se avisa	OK

Resultado de la opción 6: 10 pruebas OK, 0 fallos.

4. Evidencia de ejecución

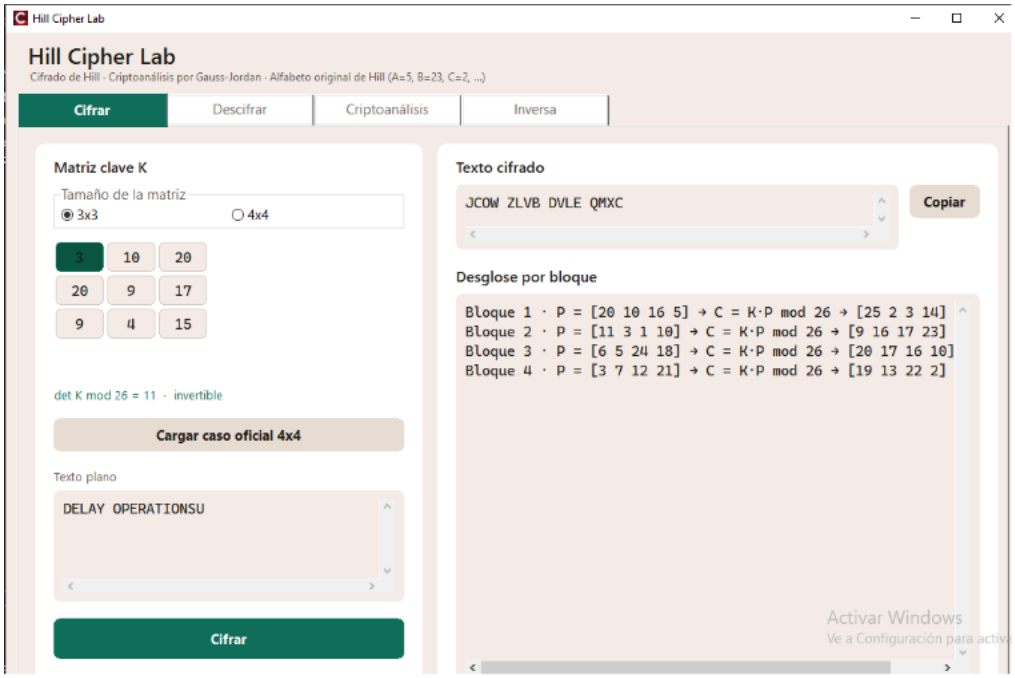
4.1 Versión de consola (aplicación principal)

Capturas de la consola (`HillConsola.exe`) que respaldan cada caso de la sección 3. Inserta cada imagen bajo su fila.

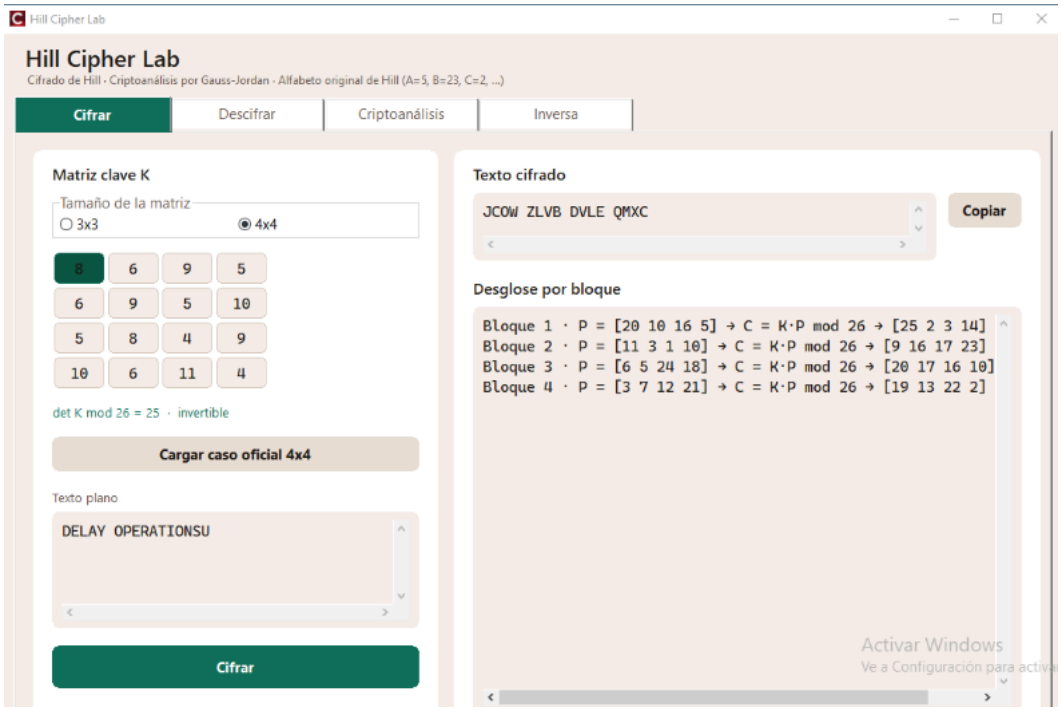
Figura	Opción del menú	Qué debe verse
1	5 – Caso oficial 4×4	Cifrado JCOWZLVBDVLEQMXC, descifrado DELAYOPERATIONSU y K recuperada = K original
2	1 – Cifrar (4×4, tecleada a mano)	K oficial, det K = 25, bloques y criptograma JCOW ZLVB DVLE QMXC
3	3 – Inversa con pasos (K 4×4)	Pasos de Gauss-Jordan (matriz aumentada inicial, pivotes, operaciones, $[I \mid K^{-1}]$) y $K \cdot K^{-1} = I$: CORRECTO
4	2 – Descifrar	Criptograma JCOWZLVBDVLEQMXC descifrado con K^{-1}
5	4 – Criptoanálisis (caso oficial)	P y C, `det(P) mod 26 = 15`, pasos de la inversión de P, K recuperada
6	4 – Criptoanálisis (texto largo 3×3)	Bloques usados 1, 4, 6 y verificación CORRECTO
7	1 – Cifrar (caso 3×3)	HOLAMUNDO → TFCQCXUNR
8	1, 2 y 3 con K singular	Mensajes de error de la tabla 3.7
9	Longitud inválida al cifrar y al descifrar	Mensajes de longitud y pregunta de relleno
10	6 – Pruebas automáticas	Las 10 pruebas en OK y «fallos: 0»

4.2 Versión con interfaz gráfica (C++Builder, VCL)

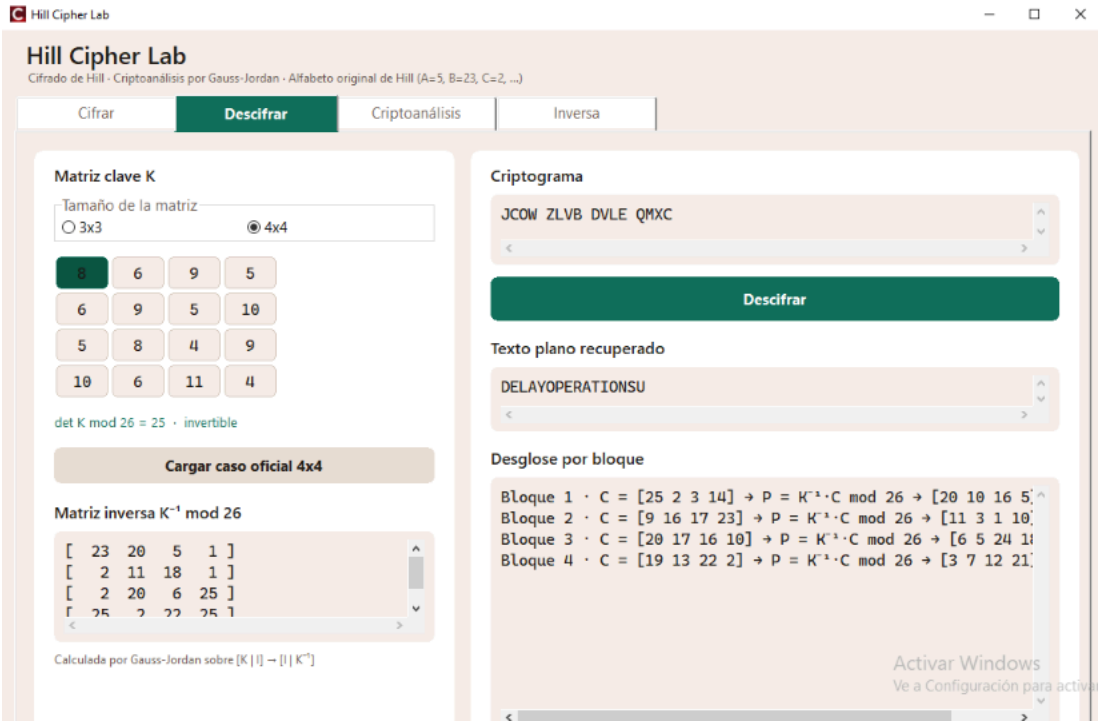
Matriz 3x3



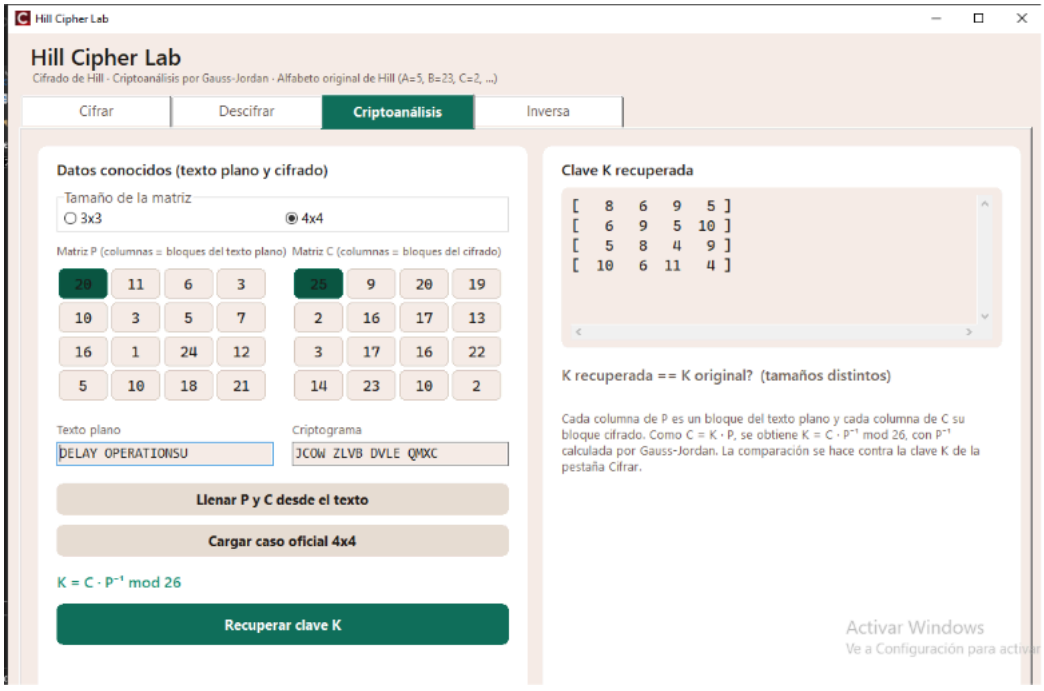
Matriz 4x4



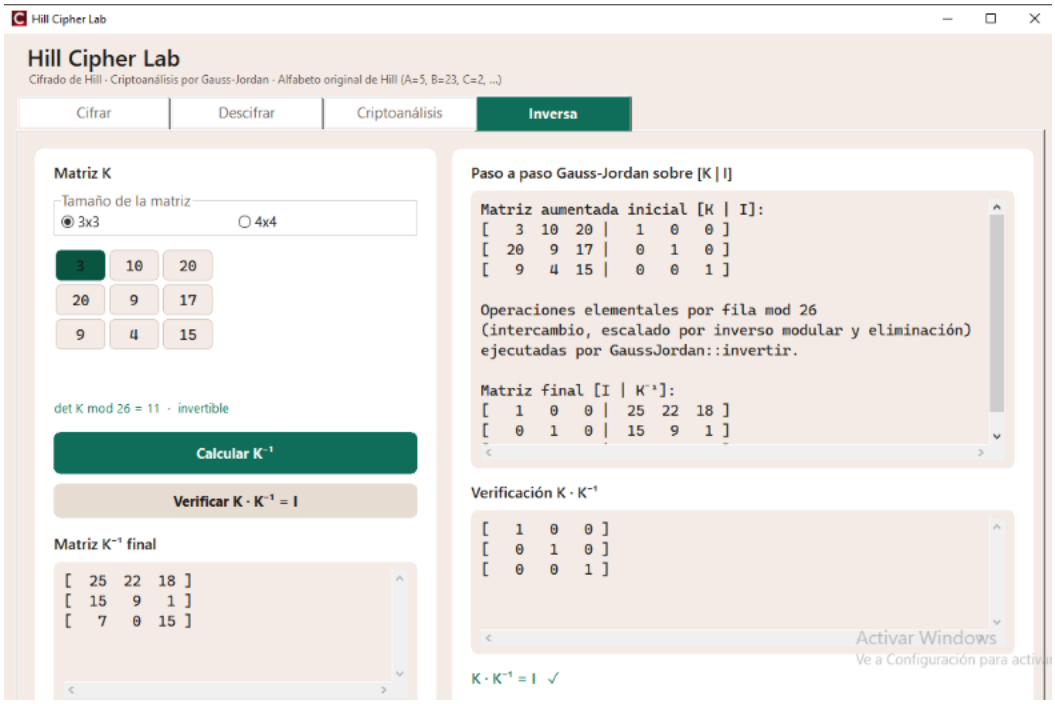
Descifrar 4x4



Critpoanalisis



Inversa



5. Reflexión sobre limitaciones y posibles mejoras

La limitación central es del algoritmo, no de la implementación: Hill es lineal y el criptoanálisis de la opción 4 lo demuestra recuperando K a partir de un par conocido.

- **Linealidad.** Con n bloques alineados de texto plano y cifrado, y P invertible mod 26, la clave completa se recupera algebraicamente (secciones 1.4 y 3.5).
- **Dependencia de que P sea invertible.** El texto HOLAMUNDO (3×3) tiene P singular, así que con 9 letras no se puede recuperar su K. La implementación mitiga esto con textos más largos probando combinaciones de bloques (hasta 100000), pero si ningún subconjunto da una P invertible el ataque falla.
- **Alfabeto no estándar.** La permutación A=5, B=23, … no añade seguridad frente a un atacante que conozca el esquema; solo dificulta usar herramientas que asuman A=0…Z=25. Además está fija en el código: una mejora sería permitir elegirla.
- **Ñ y acentos.** El texto se normaliza a A–Z; esas letras se descartan con aviso, sin convertirlas.
- **Relleno con X predecible.** Quien conozca la convención gana información sobre el último bloque. Una mejora sería indicar la longitud real del mensaje aparte.
- **Tamaño de bloque.** El menú ofrece 3×3 y 4×4, como pide el enunciado. Bloques mayores exigirían más bloques conocidos para el ataque, pero complican el cálculo de determinantes y adjuntas.

- **Cifrar con K no invertible.** El programa lo rechaza, porque el mensaje no podría descifrarse.
- **Interfaz.** La consola cubre la validación en vivo; una interfaz gráfica sería una mejora de usabilidad sin cambiar el núcleo.

6. Conclusiones

El proyecto implementa el cifrado de Hill y su criptoanálisis por Gauss-Jordan, y reproduce el caso de prueba oficial: DELAYOPERATIONSU se cifra como JCOWZLVBDVLEQMXC, se descifra al original, se invierte K con 23 pasos verificables y el ataque de texto plano conocido recupera la misma K.

El enunciado no especifica el alfabeto ni la clave; se usó el alfabeto original de Hill porque reproduce el criptograma esperado. Con ese alfabeto det K = 25 (coprimo con 26), distinto del 21 que menciona el enunciado, diferencia que se documenta en la sección 3.1.

El núcleo es autocontenido, usa solo la librería estándar de C++ y está separado del menú de consola, lo que cumple la restricción de no usar librerías que resuelvan el algoritmo. Las limitaciones identificadas (linealidad, dependencia de que P sea invertible, relleno predecible, alfabeto fijo) son en su mayoría propiedades del cifrado de Hill y se analizan en la sección 5.

7. Referencias

- Hill, L. S. (1929). Cryptography in an Algebraic Alphabet. The American Mathematical Monthly, 36(6), 306–312.
- Stinson, D. R., & Paterson, M. (2018). Cryptography: Theory and Practice (4.ª ed.). CRC Press.
- Guía del Proyecto 2 — Cifrador de Hill y Criptoanálisis Gauss-Jordan, asignatura ELC107 Criptografía y Seguridad (Grupo D).
- Código fuente de HillConsola (`main.cpp`, `core/`, `build.bat`) del equipo.

Anexos: trazas completas de Gauss-Jordan

Trazas generadas por el programa: opción 3 para los anexos A, C y D, y opción 4 (inversión de P) para el anexo B. Se transcriben sin cambios, salvo que en los pasos de elección de pivote se omite la matriz, que es idéntica a la del paso anterior.

Anexo A. Inversa de la K oficial 4×4 (23 pasos)

```
K:  [  8  6  9  5 ]   det(K) mod 26 = 25 (invertible, det^-1 = 25)
    [  6  9  5 10 ]
    [  5  8  4  9 ]
    [ 10  6 11  4 ]

Paso 1: Matriz aumentada inicial [A | I]
  [  8  6  9  5 |  1  0  0  0 ]
  [  6  9  5 10 |  0  1  0  0 ]
  [  5  8  4  9 |  0  0  1  0 ]
  [ 10  6 11  4 |  0  0  0  1 ]
Paso 2: Columna 1: pivote = 5 (fila 3), coprimo con 26; inverso modular = 21 (5 * 21 =
1 mod 26)
Paso 3: F1 <-> F3
  [  5  8  4  9 |  0  0  1  0 ]
  [  6  9  5 10 |  0  1  0  0 ]
  [  8  6  9  5 |  1  0  0  0 ]
  [ 10  6 11  4 |  0  0  0  1 ]
Paso 4: F1 = 21 * F1 mod 26
  [  1 12  6  7 |  0  0 21  0 ]
  [  6  9  5 10 |  0  1  0  0 ]
  [  8  6  9  5 |  1  0  0  0 ]
  [ 10  6 11  4 |  0  0  0  1 ]
Paso 5: F2 = F2 - 6 * F1 mod 26
  [  1 12  6  7 |  0  0 21  0 ]
  [  0 15 21 20 |  0  1  4  0 ]
  [  8  6  9  5 |  1  0  0  0 ]
  [ 10  6 11  4 |  0  0  0  1 ]
Paso 6: F3 = F3 - 8 * F1 mod 26
  [  1 12  6  7 |  0  0 21  0 ]
  [  0 15 21 20 |  0  1  4  0 ]
  [  0 14 13  1 |  1  0 14  0 ]
  [ 10  6 11  4 |  0  0  0  1 ]
Paso 7: F4 = F4 - 10 * F1 mod 26
  [  1 12  6  7 |  0  0 21  0 ]
  [  0 15 21 20 |  0  1  4  0 ]
  [  0 14 13  1 |  1  0 14  0 ]
  [  0 16  3 12 |  0  0 24  1 ]
Paso 8: Columna 2: pivote = 15 (fila 2), coprimo con 26; inverso modular = 7 (15 * 7 =
1 mod 26)
Paso 9: F2 = 7 * F2 mod 26
  [  1 12  6  7 |  0  0 21  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0 14 13  1 |  1  0 14  0 ]
  [  0 16  3 12 |  0  0 24  1 ]
Paso 10: F1 = F1 - 12 * F2 mod 26
  [  1  0 10 17 |  0 20 23  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0 14 13  1 |  1  0 14  0 ]
  [  0 16  3 12 |  0  0 24  1 ]
Paso 11: F3 = F3 - 14 * F2 mod 26
  [  1  0 10 17 |  0 20 23  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0  0  9 17 |  1  6 12  0 ]
  [  0 16  3 12 |  0  0 24  1 ]
Paso 12: F4 = F4 - 16 * F2 mod 26
  [  1  0 10 17 |  0 20 23  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0  0  9 17 |  1  6 12  0 ]
  [  0  0 17  8 |  0 18 18  1 ]
Paso 13: Columna 3: pivote = 9 (fila 3), coprimo con 26; inverso modular = 3 (9 * 3 =
1 mod 26)
```

```
Paso 14: F3 = 3 * F3 mod 26
  [  1  0 10 17 |  0 20 23  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0 17  8 |  0 18 18  1 ]
Paso 15: F1 = F1 - 10 * F3 mod 26
  [  1  0  0  1 | 22 22  1  0 ]
  [  0  1 17 10 |  0  7  2  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0 17  8 |  0 18 18  1 ]
Paso 16: F2 = F2 - 17 * F3 mod 26
  [  1  0  0  1 | 22 22  1  0 ]
  [  0  1  0  1 |  1 13 14  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0 17  8 |  0 18 18  1 ]
Paso 17: F4 = F4 - 17 * F3 mod 26
  [  1  0  0  1 | 22 22  1  0 ]
  [  0  1  0  1 |  1 13 14  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0  0 25 |  1 24  4  1 ]
Paso 18: Columna 4: pivote = 25 (fila 4), coprimo con 26; inverso modular = 25 (25 *
25 = 1 mod 26)
Paso 19: F4 = 25 * F4 mod 26
  [  1  0  0  1 | 22 22  1  0 ]
  [  0  1  0  1 |  1 13 14  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0  0  1 | 25  2 22 25 ]
Paso 20: F1 = F1 - 1 * F4 mod 26
  [  1  0  0  0 | 23 20  5  1 ]
  [  0  1  0  1 |  1 13 14  0 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0  0  1 | 25  2 22 25 ]
Paso 21: F2 = F2 - 1 * F4 mod 26
  [  1  0  0  0 | 23 20  5  1 ]
  [  0  1  0  0 |  2 11 18  1 ]
  [  0  0  1 25 |  3 18 10  0 ]
  [  0  0  0  1 | 25  2 22 25 ]
Paso 22: F3 = F3 - 25 * F4 mod 26
  [  1  0  0  0 | 23 20  5  1 ]
  [  0  1  0  0 |  2 11 18  1 ]
  [  0  0  1  0 |  2 20  6 25 ]
  [  0  0  0  1 | 25  2 22 25 ]
Paso 23: Matriz final [I | A^-1]
  [  1  0  0  0 | 23 20  5  1 ]
  [  0  1  0  0 |  2 11 18  1 ]
  [  0  0  1  0 |  2 20  6 25 ]
  [  0  0  0  1 | 25  2 22 25 ]

Verificacion K * K^-1 mod 26 = I (4x4)  ->  CORRECTO
```

En los pasos 2, 8, 13 y 18 (elección de pivote) la matriz mostrada es la misma del paso anterior.

Anexo B. Inversión de P en el criptoanálisis del caso oficial (23 pasos)

```
P (bloques de texto plano como columnas):  det(P) mod 26 = 15 (det^-1 = 7)

Paso 1: Matriz aumentada inicial [A | I]
  [ 20 11  6  3 |  1  0  0  0 ]
  [ 10  3  5  7 |  0  1  0  0 ]
  [ 16  1 24 12 |  0  0  1  0 ]
  [  5 10 18 21 |  0  0  0  1 ]
Paso 2: Columna 1: pivote = 5 (fila 4), coprimo con 26; inverso modular = 21 (5 * 21 =
1 mod 26)
Paso 3: F1 <-> F4
  [  5 10 18 21 |  0  0  0  1 ]
  [ 10  3  5  7 |  0  1  0  0 ]
  [ 16  1 24 12 |  0  0  1  0 ]
  [ 20 11  6  3 |  1  0  0  0 ]
Paso 4: F1 = 21 * F1 mod 26
  [  1  2 14 25 |  0  0  0 21 ]
  [ 10  3  5  7 |  0  1  0  0 ]
  [ 16  1 24 12 |  0  0  1  0 ]
  [ 20 11  6  3 |  1  0  0  0 ]
Paso 5: F2 = F2 - 10 * F1 mod 26
  [  1  2 14 25 |  0  0  0 21 ]
  [  0  9 21 17 |  0  1  0 24 ]
  [ 16  1 24 12 |  0  0  1  0 ]
  [ 20 11  6  3 |  1  0  0  0 ]
Paso 6: F3 = F3 - 16 * F1 mod 26
  [  1  2 14 25 |  0  0  0 21 ]
  [  0  9 21 17 |  0  1  0 24 ]
  [  0 21  8  2 |  0  0  1  2 ]
  [ 20 11  6  3 |  1  0  0  0 ]
Paso 7: F4 = F4 - 20 * F1 mod 26
  [  1  2 14 25 |  0  0  0 21 ]
  [  0  9 21 17 |  0  1  0 24 ]
  [  0 21  8  2 |  0  0  1  2 ]
  [  0 23 12 23 |  1  0  0 22 ]
Paso 8: Columna 2: pivote = 9 (fila 2), coprimo con 26; inverso modular = 3 (9 * 3 = 1
mod 26)
Paso 9: F2 = 3 * F2 mod 26
  [  1  2 14 25 |  0  0  0 21 ]
  [  0  1 11 25 |  0  3  0 20 ]
  [  0 21  8  2 |  0  0  1  2 ]
  [  0 23 12 23 |  1  0  0 22 ]
Paso 10: F1 = F1 - 2 * F2 mod 26
  [  1  0 18  1 |  0 20  0  7 ]
  [  0  1 11 25 |  0  3  0 20 ]
  [  0 21  8  2 |  0  0  1  2 ]
  [  0 23 12 23 |  1  0  0 22 ]
Paso 11: F3 = F3 - 21 * F2 mod 26
  [  1  0 18  1 |  0 20  0  7 ]
  [  0  1 11 25 |  0  3  0 20 ]
  [  0  0 11 23 |  0 15  1 24 ]
  [  0 23 12 23 |  1  0  0 22 ]
Paso 12: F4 = F4 - 23 * F2 mod 26
  [  1  0 18  1 |  0 20  0  7 ]
  [  0  1 11 25 |  0  3  0 20 ]
  [  0  0 11 23 |  0 15  1 24 ]
  [  0  0 19 20 |  1  9  0  4 ]
Paso 13: Columna 3: pivote = 11 (fila 3), coprimo con 26; inverso modular = 19 (11 *
19 = 1 mod 26)
Paso 14: F3 = 19 * F3 mod 26
  [  1  0 18  1 |  0 20  0  7 ]
  [  0  1 11 25 |  0  3  0 20 ]
```

```
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0 19 20 |  1  9  0  4 ]
Paso 15: F1 = F1 - 18 * F3 mod 26
  [  1  0  0 13 |  0 12 22 15 ]
  [  0  1 11 25 |  0  3  0 20 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0 19 20 |  1  9  0  4 ]
Paso 16: F2 = F2 - 11 * F3 mod 26
  [  1  0  0 13 |  0 12 22 15 ]
  [  0  1  0  2 |  0 14 25 22 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0 19 20 |  1  9  0  4 ]
Paso 17: F4 = F4 - 19 * F3 mod 26
  [  1  0  0 13 |  0 12 22 15 ]
  [  0  1  0  2 |  0 14 25 22 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0  0 11 |  1  2  3 24 ]
Paso 18: Columna 4: pivote = 11 (fila 4), coprimo con 26; inverso modular = 19 (11 *
19 = 1 mod 26)
Paso 19: F4 = 19 * F4 mod 26
  [  1  0  0 13 |  0 12 22 15 ]
  [  0  1  0  2 |  0 14 25 22 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0  0  1 | 19 12  5 14 ]
Paso 20: F1 = F1 - 13 * F4 mod 26
  [  1  0  0  0 | 13 12  9 15 ]
  [  0  1  0  2 |  0 14 25 22 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0  0  1 | 19 12  5 14 ]
Paso 21: F2 = F2 - 2 * F4 mod 26
  [  1  0  0  0 | 13 12  9 15 ]
  [  0  1  0  0 | 14 16 15 20 ]
  [  0  0  1 21 |  0 25 19 14 ]
  [  0  0  0  1 | 19 12  5 14 ]
Paso 22: F3 = F3 - 21 * F4 mod 26
  [  1  0  0  0 | 13 12  9 15 ]
  [  0  1  0  0 | 14 16 15 20 ]
  [  0  0  1  0 | 17  7 18  6 ]
  [  0  0  0  1 | 19 12  5 14 ]
Paso 23: Matriz final [I | A^-1]
  [  1  0  0  0 | 13 12  9 15 ]
  [  0  1  0  0 | 14 16 15 20 ]
  [  0  0  1  0 | 17  7 18  6 ]
  [  0  0  0  1 | 19 12  5 14 ]

K recuperada = C * P^-1:
  [  8  6  9  5 ]
  [  6  9  5 10 ]
  [  5  8  4  9 ]
  [ 10  6 11  4 ]
K recuperada == K original? SI
```

En los pasos 2, 8, 13 y 18 (elección de pivote) la matriz mostrada es la misma del paso anterior.

Anexo C. Inversa del caso 3×3 (13 pasos)

```
K:  [  3 10 20 ]   det(K) mod 26 = 11 (invertible, det^-1 = 19)
    [ 20  9 17 ]
    [  9  4 15 ]

Paso 1: Matriz aumentada inicial [A | I]
  [  3 10 20 |  1  0  0 ]
  [ 20  9 17 |  0  1  0 ]
  [  9  4 15 |  0  0  1 ]
Paso 2: Columna 1: pivote = 3 (fila 1), coprimo con 26; inverso modular = 9 (3 * 9 = 1
mod 26)
Paso 3: F1 = 9 * F1 mod 26
  [  1 12 24 |  9  0  0 ]
  [ 20  9 17 |  0  1  0 ]
  [  9  4 15 |  0  0  1 ]
Paso 4: F2 = F2 - 20 * F1 mod 26
  [  1 12 24 |  9  0  0 ]
  [  0  3  5 |  2  1  0 ]
  [  9  4 15 |  0  0  1 ]
Paso 5: F3 = F3 - 9 * F1 mod 26
  [  1 12 24 |  9  0  0 ]
  [  0  3  5 |  2  1  0 ]
  [  0  0  7 | 23  0  1 ]
Paso 6: Columna 2: pivote = 3 (fila 2), coprimo con 26; inverso modular = 9 (3 * 9 = 1
mod 26)
Paso 7: F2 = 9 * F2 mod 26
  [  1 12 24 |  9  0  0 ]
  [  0  1 19 | 18  9  0 ]
  [  0  0  7 | 23  0  1 ]
Paso 8: F1 = F1 - 12 * F2 mod 26
  [  1  0  4 |  1 22  0 ]
  [  0  1 19 | 18  9  0 ]
  [  0  0  7 | 23  0  1 ]
Paso 9: Columna 3: pivote = 7 (fila 3), coprimo con 26; inverso modular = 15 (7 * 15 =
1 mod 26)
Paso 10: F3 = 15 * F3 mod 26
  [  1  0  4 |  1 22  0 ]
  [  0  1 19 | 18  9  0 ]
  [  0  0  1 |  7  0 15 ]
Paso 11: F1 = F1 - 4 * F3 mod 26
  [  1  0  0 | 25 22 18 ]
  [  0  1 19 | 18  9  0 ]
  [  0  0  1 |  7  0 15 ]
Paso 12: F2 = F2 - 19 * F3 mod 26
  [  1  0  0 | 25 22 18 ]
  [  0  1  0 | 15  9  1 ]
  [  0  0  1 |  7  0 15 ]
Paso 13: Matriz final [I | A^-1]
  [  1  0  0 | 25 22 18 ]
  [  0  1  0 | 15  9  1 ]
  [  0  0  1 |  7  0 15 ]

Verificacion K * K^-1 mod 26 = I (3x3)  ->  CORRECTO
```

En los pasos 2, 6 y 9 (elección de pivote) la matriz mostrada es la misma del paso anterior.

Anexo D. Matriz singular: K = [[1,2,3],[2,4,6],[5,7,9]] (opción 3)

```
det(K) mod 26 = 0
K NO es invertible mod 26 (mcd(det,26) != 1).

Paso 1: Matriz aumentada inicial [A | I]
  [  1  2  3 |  1  0  0 ]
  [  2  4  6 |  0  1  0 ]
  [  5  7  9 |  0  0  1 ]
Paso 2: Columna 1: pivote = 1 (fila 1), coprimo con 26; inverso modular = 1 (1 * 1 = 1
mod 26)
Paso 3: F2 = F2 - 2 * F1 mod 26
  [  1  2  3 |  1  0  0 ]
  [  0  0  0 | 24  1  0 ]
  [  5  7  9 |  0  0  1 ]
Paso 4: F3 = F3 - 5 * F1 mod 26
  [  1  2  3 |  1  0  0 ]
  [  0  0  0 | 24  1  0 ]
  [  0 23 20 | 21  0  1 ]
Paso 5: Columna 2: pivote = 23 (fila 3), coprimo con 26; inverso modular = 17 (23 * 17
= 1 mod 26)
Paso 6: F2 <-> F3
  [  1  2  3 |  1  0  0 ]
  [  0 23 20 | 21  0  1 ]
  [  0  0  0 | 24  1  0 ]
Paso 7: F2 = 17 * F2 mod 26
  [  1  2  3 |  1  0  0 ]
  [  0  1  2 | 19  0 17 ]
  [  0  0  0 | 24  1  0 ]
Paso 8: F1 = F1 - 2 * F2 mod 26
  [  1  0 25 | 15  0 18 ]
  [  0  1  2 | 19  0 17 ]
  [  0  0  0 | 24  1  0 ]
Paso 9: Columna 3: ningun elemento (desde la fila 3) es coprimo con 26; A no es
invertible mod 26

K no tiene inversa mod 26; no hay nada que verificar.
```

La columna 3 no tiene ningún elemento coprimo con 26 desde la fila 3 (el único candidato es 0), por lo que el algoritmo se detiene y declara que K no es invertible.